

SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3



Engineer to Excel



SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3



Engineer to Excel



SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



Code of Conduct:

I Dhaanush Rajan (192421162) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Testing as an Engineering Activity

Testing as an engineering activity means testing is performed in a systematic, planned and measurable manner, rather than randomly.

Approach:

1. Analyze requirements.
2. Identify test conditions.
3. Design test cases systematically.
4. Define expected results.
5. Execute and document tests.
6. Report and track defects.
7. Perform regression testing.

Example: For a login system, test valid credentials, invalid credentials, empty fields, incorrect password and account lockout.

Benefits:

- Better test coverage
- Early defect detection
- Consistent results
- Reduced testing cost
- Improved software quality

2. Well-Defined Testing Process

A well-defined testing process provides a standard sequence of activities that testers follow from the beginning to the end of a project. It ensures that important testing activities are not skipped.

The major stages are:

Requirement Analysis → Test Planning → Test Design → Test Execution → Defect Reporting → Retesting → Regression Testing → Test Closure.

During requirement analysis, testers understand the expected system behavior. In test planning, the scope, resources, schedule, tools, and risks are decided. Test cases and test data are prepared during test design. During execution, actual results are compared with expected results. Any mismatch is reported as a defect. After the developer fixes it, the tester performs retesting. Regression testing is then performed to ensure that the fix has not affected other functions.

For example, in a banking application, predefined test cases can be created for cash withdrawal, deposit, balance enquiry, and fund transfer. Each activity is tested according to the planned process.

A well-defined process provides consistent testing, better coverage, easier defect tracking, reduced testing errors, and improved software quality. Thus, a standard process makes testing more reliable and manageable.

3. Testing in SDLC

Testing should be performed throughout the Software Development Life Cycle (SDLC).

- Requirement phase: Review requirements.
- Design phase: Review architecture and interfaces.
- Coding phase: Perform unit testing.
- Integration phase: Test communication between modules.
- System phase: Test the complete system.
- Acceptance phase: Validate business requirements.
- Maintenance: Perform regression testing.

Example: In an e-commerce system, login, cart, payment and order modules are tested individually and together.

Result: Structured testing reduces defects and improves software quality.

4. Error, Defect and Failure

These three terms represent different stages of a problem.

- Error: Human mistake made during development.
- Defect: Fault introduced into the software due to an error.
- Failure: Incorrect behavior observed during execution.

Example:

Developer misunderstands the password requirement → Error

Incorrect password validation is coded → Defect

User enters a valid password but login fails → Failure

Therefore:

Error → Defect → Failure

Understanding these terms helps testers and developers identify and resolve problems correctly.

5. Early Defect Detection

The testing principle “Testing should start early” helps detect defects before they become expensive.

Testers should participate during:

- Requirement review
- Design review
- Test planning
- Test-case preparation

Example: If a banking requirement incorrectly specifies interest calculation, identifying it during requirement review is cheaper than fixing it after implementation.

Benefits:

- Reduces development cost
- Reduces rework
- Detects ambiguity early
- Saves time
- Improves quality

6. Tester Involvement in Requirement Phase

Testers should participate during requirement analysis to identify problems before coding begins.

They check for:

- Ambiguous requirements
- Missing requirements
- Contradictory requirements
- Unclear input conditions
- Missing validation rules

Example: In a healthcare system, if a requirement says patient data must be stored “for a long period,” the tester should ask for the exact retention period.

Benefits:

- Prevents requirement defects
- Improves testability
- Reduces rework
- Saves cost
- Improves final product quality

7. Explain How Unclear Requirements Cause Defects

Requirements define what the software should do. If requirements are unclear, developers may make different assumptions, resulting in defects.

Defect Flow

Unclear Requirement → Wrong Design → Wrong Implementation → Testing Failure → Production Defect

Types of Requirement Problems

1. **Ambiguous requirements**
2. **Incomplete requirements**
3. **Contradictory requirements**
4. **Missing business rules**
5. **Undefined input conditions**
6. **Unclear expected outputs**
7. **Non-testable requirements**

Example

Consider a payroll application.

Suppose the requirement says:

“Employees should receive overtime payment according to company policy.”

However, it does not specify:

- Overtime rate.
- Maximum overtime hours.
- Eligible employees.
- Weekend overtime rules.
- Holiday overtime rules.

Different developers may interpret the requirement differently.

One developer may calculate overtime as 1.5 times the salary, while another may use 2 times the salary.

This creates incorrect results and defects.

Prevention

Unclear requirements can be prevented through:

- Requirement reviews.
- Stakeholder discussions.
- Prototyping.
- Acceptance criteria.
- Requirement traceability.
- Clear business rules.
- Testability reviews.

Benefits of Clear Requirements

- Reduces development errors.
- Reduces testing failures.
- Reduces rework.
- Improves customer satisfaction.
- Makes test design easier.
- Improves software quality.

Conclusion:

Requirement quality directly affects software quality. Therefore, testers should identify and clarify unclear requirements **before development begins**.

8. Explain Boundary Value Analysis with an Example

Boundary Value Analysis (BVA) is a black-box testing technique in which test cases are selected around the boundaries of an input range.

Why These Values?

Suppose the developer accidentally writes:

Age > 18

instead of:



Engineer to Excel



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



Age ≥ 18

The value **18** would incorrectly be rejected.

Similarly, if the maximum condition is incorrectly implemented, testing **60 and 61** can identify the problem.

Applications

BVA can be used for:

- Age limits.
- Transaction amounts.
- Password length.
- File size.
- Number of items.
- Marks.
- Salary ranges.
- Date ranges.

Advantages

1. Detects boundary-related defects.
2. Finds off-by-one errors.
3. Requires relatively few test cases.
4. Improves input validation testing.
5. Useful for numerical limits.
6. Helps identify incorrect minimum and maximum conditions.

Example in Banking

If a bank allows transactions from ₹100 to ₹50,000:

- ₹99 → Invalid
- ₹100 → Valid
- ₹101 → Valid
- ₹49,999 → Valid
- ₹50,000 → Valid
- ₹50,001 → Invalid

Conclusion:

Boundary Value Analysis is an effective technique for detecting defects at the **edges of valid and invalid input ranges**.

9. Explain Defect Repository and Its Importance

A **defect repository** is a centralized system used to record, track, manage, and analyze software defects throughout the project.

It provides a complete history of defects and their resolutions.

Information Stored in a Defect Repository

A typical defect record contains:

1. Defect ID
2. Defect title
3. Description
4. Module or feature
5. Steps to reproduce
6. Expected result
7. Actual result
8. Severity
9. Priority
10. Environment
11. Assigned developer
12. Status
13. Root cause
14. Resolution
15. Retesting result

Example

Suppose an online payment application displays an error after correct card details are entered.

The tester records:

- **Defect:** Payment fails with valid card.

- **Steps:** Login → Select product → Enter valid card → Pay.
- **Expected:** Payment should succeed.
- **Actual:** Payment error displayed.
- **Severity:** High.
- **Status:** Open.

After the developer fixes it, the tester retests the defect.

Importance

1. Easy Tracking

Every defect can be monitored from discovery to closure.

2. Avoids Duplicate Defects

Testers can check whether the same problem has already been reported.

3. Supports Root-Cause Analysis

Historical information helps identify recurring causes.

4. Supports Regression Testing

Important previous defects can become permanent regression test cases.

5. Provides Historical Data

Past defects help identify high-risk modules.

6. Improves Communication

Developers receive clear information about the problem.

Conclusion:

A defect repository makes defect management **systematic, transparent, and efficient**, while also providing valuable information for future testing.

10. Explain Functional Defects in an ATM System

A **functional defect** occurs when a software function does not behave according to its specified requirement.

ATM systems handle financial transactions, so functional defects can cause serious problems.

Example: Cash Withdrawal

Suppose a customer requests ₹2,000.

The correct behavior should be:

1. Verify customer authentication.
2. Check account balance.
3. Verify withdrawal limit.
4. Dispense ₹2,000.
5. Deduct ₹2,000 from the account.
6. Display successful transaction.
7. Generate transaction record.

If the ATM deducts ₹2,000 but does not dispense cash, a functional defect has occurred.

Test Scenarios

Positive Testing

- Valid card.
- Correct PIN.
- Sufficient balance.
- Valid withdrawal amount.

Negative Testing

- Invalid PIN.
- Blocked card.
- Insufficient balance.
- Invalid amount.
- Invalid account.

Boundary Testing

- Minimum withdrawal.
- Maximum withdrawal.
- Amount just below limit.
- Amount just above limit.

Integration Testing

- ATM and bank server.
- ATM and account database.
- Transaction and notification system.

Possible Functional Defects

- Wrong amount dispensed.
- Incorrect balance deduction.
- Transaction completed without cash.
- Cash dispensed twice.
- Incorrect transaction status.
- Incorrect receipt.
- Incorrect transaction history.

Importance

Proper ATM functional testing:

- Prevents financial loss.
- Protects customer accounts.
- Improves reliability.
- Reduces transaction failures.
- Increases customer trust.
- Ensures business requirements are satisfied.

Conclusion:

Functional testing of ATM systems must cover **normal, abnormal, boundary, and integration scenarios** to ensure accurate and secure financial transactions.

11. Defect Clustering

Defect clustering means that a small number of modules often contain a large percentage of defects.

Example:

Login – 5 defects

Payment – 40 defects

Profile – 6 defects

Search – 4 defects

Here, the payment module should receive more testing attention.

Benefits:

- Focuses testing on high-risk areas
- Uses resources efficiently
- Finds more defects
- Reduces testing time
- Improves overall quality

Thus, defect clustering helps prioritize testing activities.

12. Performance Defect

Slow response during heavy user load is classified as a performance defect.

Performance testing can include:

- Load testing
- Stress testing
- Volume testing
- Response-time testing
- Concurrent-user testing

Example: A website works properly with 100 users but takes 20 seconds to respond when 10,000 users access it.

Classification helps the team select appropriate testing methods and identify performance bottlenecks.

13. Historical Defect Data in Regression Testing

Historical defect data can be reused to improve regression testing.

Steps:

1. Collect previous defect reports.
2. Identify frequently failing modules.
3. Convert important defects into test cases.
4. Add them to the regression suite.
5. Execute them after every release.

Example: If previous versions had payment failures, payment test cases should always be included in regression testing.

Benefits:

- Prevents defect recurrence
- Improves coverage
- Saves test-design time
- Focuses on high-risk areas
- Improves release quality

14. Tester Role in Integration Testing

Testers help reduce integration defects by collaborating with developers early.

Their responsibilities include:

- Understanding module interfaces
- Reviewing data flow
- Designing integration test cases
- Testing APIs and interfaces
- Reporting defects clearly

Example: If the payment module sends an incorrect transaction ID to the order module, integration testing can detect it.

Early tester involvement improves communication and reduces integration failures.

15. Defect Prevention

Defect prevention focuses on removing the root cause of defects instead of repeatedly fixing the same problem.

Steps:

Identify recurring defect → Perform root-cause analysis → Correct process → Implement preventive action → Monitor

Example: If database validation defects occur in every release, the team can improve coding standards, reviews and validation practices.

Benefits:

- Reduces recurring defects
- Reduces maintenance cost
- Improves development process
- Saves time
- Improves software quality

16. Structured Testing for Mobile Applications

A mobile application requires a standardized testing strategy.

Testing should include:

- Functional testing
- UI testing
- Compatibility testing
- Performance testing
- Network testing
- Device testing
- OS-version testing
- Regression testing

Example: Test the application on different Android versions, screen sizes and network conditions.

A structured approach produces consistent test results and improves application reliability.

17. Testing vs Debugging

Testing and debugging have different purposes.

Testing:

- Finds failures or defects.
- Mainly performed by testers.
- Checks whether software works correctly.
- Reports defects.

Debugging:

- Finds and fixes causes of defects.
- Mainly performed by developers.
- Investigates why the software failed.
- Corrects the code.

Example: If an ATM does not dispense cash, testing identifies and reports the problem. Debugging investigates the transaction code and fixes it.

Thus, testing detects problems while debugging fixes their causes.

18. Cross-Browser Testing

Cross-browser testing verifies that a web application works consistently across different browsers.

The team should test browsers such as:

- Chrome

- Edge
- Firefox
- Safari

Test:

- Login
- Forms
- Product display
- Shopping cart
- Payment
- JavaScript
- Responsive layout

Example: If checkout works in Chrome but fails in Firefox, compatibility testing identifies the problem.

This improves user experience and browser compatibility.

19. Validation in Logistics Application

Improper validation can cause inconsistent outputs.

Testers should check:

- Valid inputs
- Invalid inputs
- Missing values
- Boundary values
- Incorrect formats
- Duplicate values

Example: A logistics system should reject an invalid shipment ID or delivery date.

Proper validation ensures that only correct data enters the system and improves reliability.

20. Systematic Test Design

Random testing provides unpredictable and incomplete coverage.

A systematic approach includes:

Requirement Analysis → Test Conditions → Test Cases → Expected Results → Execution → Defect Reporting → Regression

Example: For registration, test valid details, empty fields, duplicate email, invalid email and boundary-length passwords.

Benefits:

- Better coverage
- Repeatable testing
- Improved defect detection
- Easier tracking
- Better quality

Therefore, systematic test design is more effective than random testing.

21. Integration Data Consistency

Data inconsistency between modules can occur because of:

- Incorrect interfaces
- Data-format mismatch
- Incorrect database mapping
- Synchronization problems
- API errors

Example: The patient module sends patient ID P1001, but the billing module receives P1010.

Integration testing should verify data passed between modules.

Benefits:

- Detects interface errors
- Ensures accurate data transfer
- Improves system reliability
- Prevents inconsistent records

22. Testers Across SDLC

Testers have roles throughout the SDLC.

- Requirements: Review requirements.
- Design: Check testability.
- Coding: Prepare test cases.
- Integration: Test module interaction.
- System: Test complete application.
- Acceptance: Validate requirements.
- Maintenance: Perform regression testing.

Early involvement allows testers to identify defects before implementation.

Result: Less rework, lower cost and better quality.

23. Testing Peak Usage

Failure during peak usage indicates insufficient performance test coverage.

The team should perform:

- Load testing
- Stress testing
- Volume testing
- Concurrent-user testing
- Peak-load testing

Example: A website tested with 100 users may crash when 10,000 users access it simultaneously.

Testing realistic peak scenarios before deployment prevents production failures and improves reliability.

24. Defect Documentation

A **defect repository** is a centralized place used to record, track, analyze and manage software defects throughout the project.

Each defect should contain:

Defect ID → Description → Steps to Reproduce → Expected Result → Actual Result → Severity → Priority → Status → Root Cause → Resolution

Example:

If a payment fails after entering correct card details, the tester records the defect with its steps, expected payment success, actual error message, severity and current status.

Benefits of Defect Documentation

1. **Easy tracking** – The team can monitor the status of every defect.
2. **Prevents duplicate defects** – Existing defects can be checked before reporting a new one.
3. **Supports root-cause analysis** – Previous defect information helps identify why defects occur.
4. **Helps regression testing** – Fixed defects can be converted into regression test cases.
5. **Provides historical information** – Past defects help improve future testing and prevent repeated problems.

Conclusion: Proper defect documentation makes defect management systematic, improves testing efficiency and increases overall software quality.

25. Banking System Testing

A banking system requires structured and rigorous testing.

Important testing includes:

- Functional testing
- Integration testing
- Security testing
- Boundary testing
- Negative testing
- Performance testing
- Regression testing

Example: Fund transfer should be tested for insufficient balance, invalid account number, duplicate transaction and network failure.

Structured testing improves reliability and reduces transaction failures.

26. Edge Case Testing

Testing only normal inputs can miss important defects.

Boundary testing should be used to test values near limits.

For a transaction limit of ₹100–₹50,000, test:

₹99, ₹100, ₹101, ₹49,999, ₹50,000, ₹50,001

This detects:

- Boundary errors
- Off-by-one errors
- Invalid limit handling
- Unexpected crashes

Therefore, edge and boundary testing improves defect detection.

27. Standardized Testing Process

A common process should be followed across all modules.

Standard Process:

Planning → Requirement Analysis → Test Design → Execution → Defect Reporting → Retesting → Regression → Closure

The team should use common:

- Test-case templates
- Defect-report formats
- Testing standards
- Review procedures

Benefits:

- Consistent testing
- Reduced errors
- Better coverage
- Easier management
- Uniform quality

28. Input Validation

Input validation ensures that the system accepts valid data and rejects invalid data.

Test cases should include:

- Empty input
- Invalid format
- Special characters
- Very long input
- Boundary values
- Duplicate values
- Unexpected input

Example: If age should be 18–60, test 17, 18, 60, 61 and non-numeric values.

Proper validation prevents crashes and improves system robustness.

29. Defect Leakage

Defect leakage occurs when defects are not detected during testing and **escape into the production environment**, where they are discovered by users or customers.

Causes of Defect Leakage

1. **Poor test coverage** – Important scenarios are not tested.
2. **Incomplete requirements** – Some requirements are missed or misunderstood.

3. **Insufficient regression testing** – Existing functionality is not rechecked after changes.
4. **Random testing** – Test cases are not designed systematically.
5. **Poor defect analysis** – Previous defects are not studied to prevent recurrence.

Solutions

- Perform proper **requirement reviews**.
- Use **systematic test-case design**.
- Apply **boundary and negative testing**.
- Perform **regression testing** after changes.
- Use **risk-based testing** for critical features.
- Maintain and analyze a **defect repository**.

Example: In a banking application, a fund-transfer defect may reach production because insufficient-balance and network-failure scenarios were not tested.

Conclusion: Improved test coverage, systematic testing and effective regression testing reduce defect leakage and improve software reliability and customer satisfaction.

30. Early Defect Detection

Early defect detection means identifying defects as early as possible in the SDLC.

The team should:

1. Review requirements.
2. Conduct design reviews.
3. Prepare test cases early.
4. Perform unit testing.
5. Perform integration testing.
6. Conduct continuous regression testing.

Example: Finding an incorrect requirement during requirement review is much cheaper than finding it after deployment.

Benefits:

- Lower cost
- Less rework
- Reduced development time
- Fewer production defects
- Better software quality

Key point: The earlier a defect is found, the cheaper and easier it is to fix.



SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

